

Database Technologies

BIS605

Software Development Technologies for Digital Business

Asst. Prof. Dr. Worarat Krathu

Resources

- <https://en.wikipedia.org/wiki/Database>
- <https://enacademic.com/dic.nsf/enwiki/334436>
- <https://www.codecademy.com/articles/what-is-rdbms-sql>
- Nimrakhn89, Relational Database & Database Management System, at slideshare.net
- Ramakant Soni, NoSQL Towards the end of RDBMS?, at slideshare.net
- Markus Klems, Apache Cassandra Lesson: Data Modelling and CQL3, at slideshare.net
- <https://nerohoop.gitbooks.io/databases/content/relational-database/scaling.html>
- <https://cassandra.apache.org/doc/latest/cql/index.html>
- <https://docs.mongodb.com/manual/tutorial/query-documents/>

Database

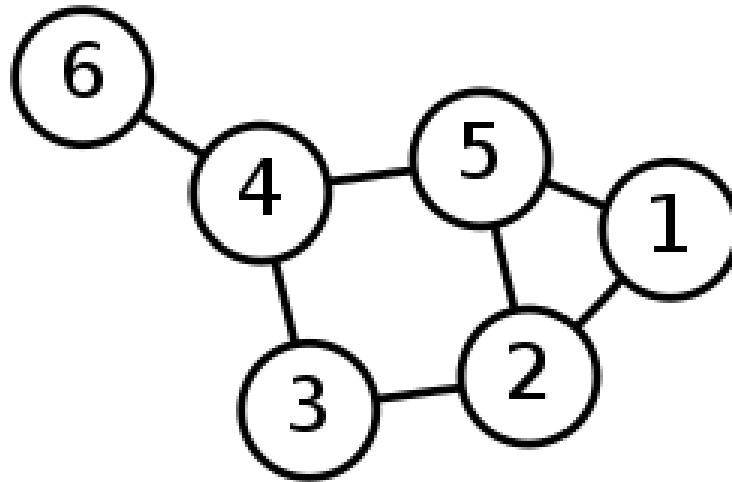
- Database is an organization of data (store, retrieve, update, delete, etc.)
- A "database" refers to a set of related data and the way it is organized
- A "database management system" (DBMS) is a software or a set of software that provides access to database
 - Data definition
 - Update
 - Retrieval
 - Administration

Database Era

- Database Era (based on data model structure)
 - Navigational
 - Relational
 - Object-oriented
 - NoSQL

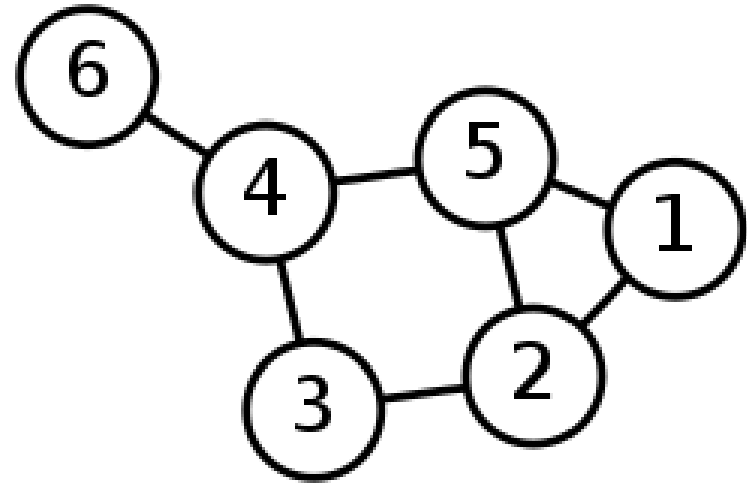
Navigational (1960s)

- Associated with network model and hierarchical model of database
- Records or objects are found primarily by following references from other objects.



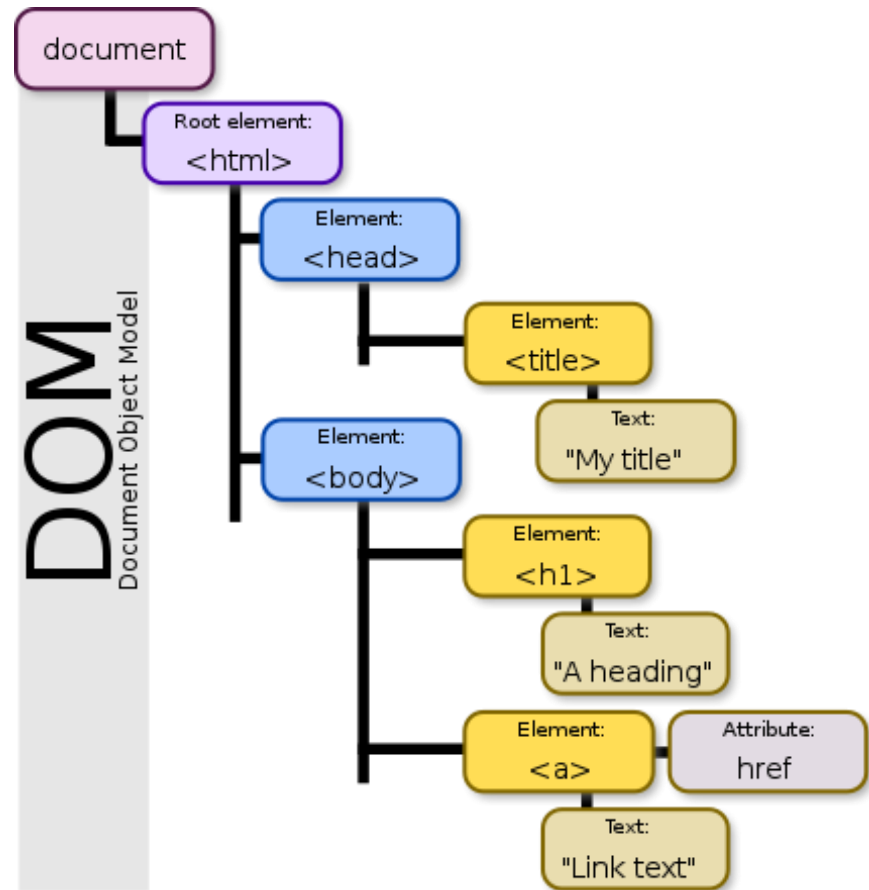
Navigational (1960s)

- Navigational APIs instruct the system in a sequence of steps *how* to reach the required records
 - Node6/Node4/Node5/Node1
 - valid
 - Node6/Node2/Node1
 - invalid



DOM and XPath

- Example - DOM
 - Used in web browsers
 - Uses Xpath as query language



DOM and XPath

- `/html/body/article[1]/div/p[1]/b`

← → ↻ ⓘ File | D:/GoogleDrive/SIT/BIS605/2563_1/Mockupweb/exercise10_solution.html

School of Information Technology

Doctoral Programs

The Doctor of Philosophy programs aim to produce highly qualified researchers and scholars to international standards. It also aims to create researchers of integrity as clearly stated in the SIT "Five P's" philosophy: Prompt actions, Punctuality, Professionalism, Perseverance, and Principles and can respond to computer science industry requirements. Moreover, the program aims for the discovery of new insights and academic excellence for national development by emphasizing in-depth research and the use of resources at maximum potential.

SIT provides two doctoral programs as follows:

- [Information Technology](#)
- [Computer Science](#)

Information Technology

Philosophy
The Doctor of Philosophy program in Information Technology aims to produce highly qualified researchers and scholars to international standards. It also aims to create researchers of integrity as clearly stated in the SIT "Five P's" philosophy: Prompt actions, Punctuality, Professionalism, Perseverance, and Principles and can respond to computer science industry requirements. Moreover, the program aims for the discovery of new insights and academic excellence for national development by emphasizing in-depth research and the use of resources at maximum potential. London is the capital city of England. It is the most populous city in the United Kingdom, with a metropolitan area of over 13 million inhabitants.

Program objectives:

1. To produce productive researchers and scholars in information technology to international standards and promote significant human resources for the betterment of society
2. To conduct original research, develop new technology in information technology, and collaborate academically with leading foreign universities
3. To achieve for an academic research excellence, not only within Thailand but also abroad through continuous research development

Computer Science

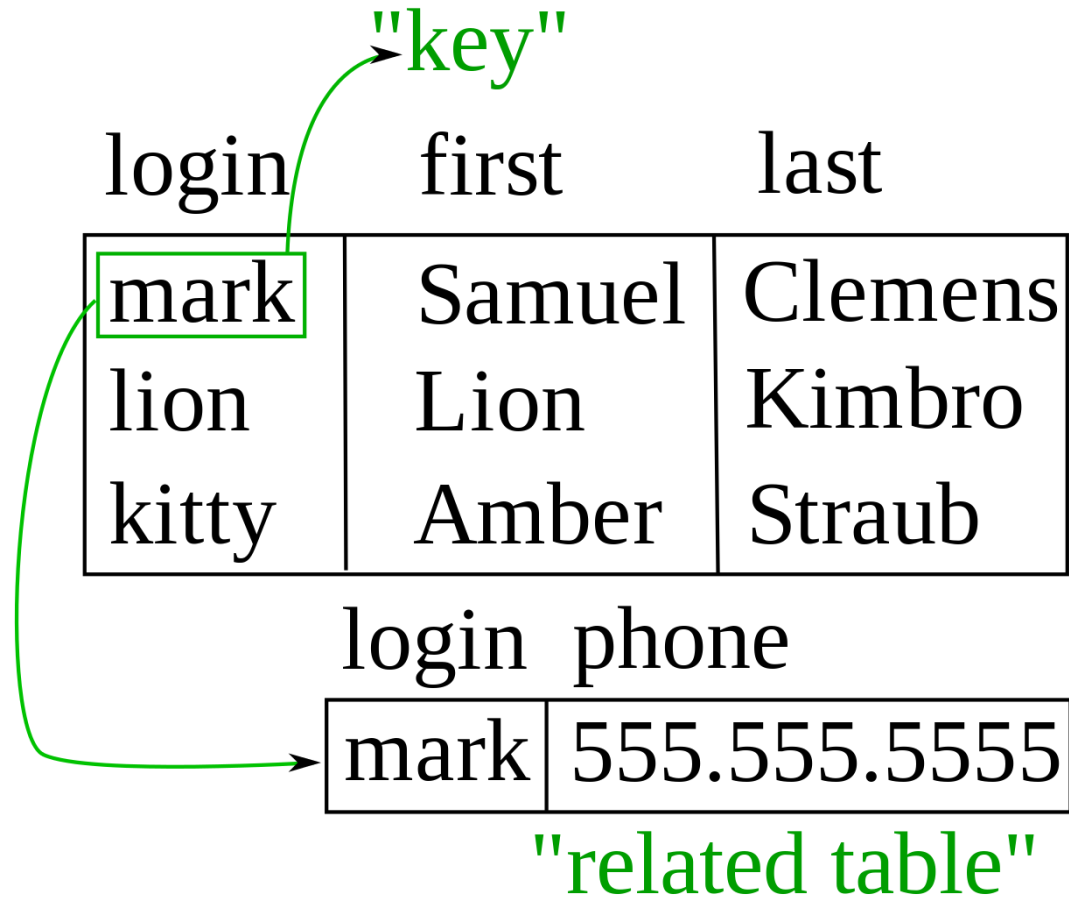
Philosophy
The Doctor of Philosophy program in Computer Science aims to produce highly qualified researchers and scholars to international standards. It also aims to create researchers of integrity as clearly stated in the SIT "Five P's" philosophy:

```
<html>
  <head>...</head>
  <body>
    <header>
      <h1 id="headTitle" onmouseover="changeHeader()" onmouseout="changeHeaderBack()" :
r: midnightblue;">
      <h2>Doctoral Programs</h2>
    </header>
    <p id="firstPara">
      "The Doctor of Philosophy programs aim to produce highly qualified researchers ai
to international standards. It also aims to create researchers of integrity as c
stated in the SIT "Five P's" philosophy: Prompt actions, Punctuality, Profession
Perseverance, and Principles and can respond to computer science industry requir
Moreover, the program aims for the discovery of new insights and academic excell
national development by emphasizing in-depth research and the use of resources a
potential."
    </p>
    <nav>
      "
      SIT provides two doctoral programs as follows:
      "
      <ul>...</ul>
    </nav>
    <article>
      <h3 id="IT">Information Technology</h3>
      <div>
        <p onmouseover="changeBackground(this)" onmouseout="changeBackgroundBack(this)"
background-color: linen;">
        <b>Philosophy</b> == $0
        <br>
        "The Doctor of Philosophy program in Information Technology aims to produce h
qualified researchers and scholars to international standards. It also aims t
researchers of integrity as clearly stated in the SIT "Five P's" philosophy:
actions, Punctuality, Professionalism, Perseverance, and Principles and can r
computer science industry requirements. Moreover, the program aims for the di
```


Relational (1970s)

- Navigational approach lacks of “search” facility
- Relational approach allows us to identify and access data in relation to another piece of data
- Often organized into tables
- Based on concept of linked list

Relational (1970s)



Relational (1970s)

- In a relational database, we can define several operations to create new relations out of the existing ones.
- Basic operations:
 - Insert
 - Delete
 - Update
 - Select
 - Join
 - Union
 - Intersection

Relational (1970s)

- Insert (tuple)

COURSES

No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS19	UNIX	4
CIS51	Networking	5

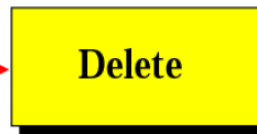


No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS19	UNIX	4
CIS51	Networking	5
<i>CIS52</i>	<i>TCP/IP Protocols</i>	<i>6</i>

- Delete (tuple)

COURSES

No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS19	UNIX	4
CIS51	Networking	5
CIS52	TCP/IP Protocols	6



No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS51	Networking	5
CIS52	TCP/IP Protocols	6

Relational (1970s)

- Update (some attributes of a tuple)

COURSES

No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS19	UNIX	4
CIS51	Networking	5
CIS52	TCP/IP Protocols	6



No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS19	UNIX	4
CIS51	Networking	6
CIS52	TCP/IP Protocols	6

- Select

COURSES

No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS19	UNIX	4
CIS51	Networking	5
CIS52	TCP/IP Protocols	6



No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS51	Networking	5

Relational (1970s)

- Join

COURSES

No	Course-Name	Unit
CIS15	Intro to C	5
CIS17	Intro to Java	5
CIS19	UNIX	4
CIS51	Networking	5
CIS52	TCP/IP Protocols	6

TAUGHT-BY

No	Professor
CIS15	Lee
CIS17	Lu
CIS19	Walter
CIS51	Lu
CIS52	Lee

Join

No	Course-Name	Unit	Professor
CIS15	Intro to C	5	Lee
CIS17	Intro to Java	5	Lu
CIS19	UNIX	4	Walter
CIS51	Networking	5	Lu
CIS52	TCP/IP Protocols	6	Lee

Relational (1970s)

- Union

CIS15-Roster

Student-ID	F-Name	L-Name
145-67-6754	John	Brown
232-56-5690	George	Yellow
345-89-6580	Anne	Green
459-98-6789	Ted	Purple

CIS52-Roster

Student-ID	F-Name	L-Name
342-88-9999	Rich	White
145-67-6754	John	Brown
232-56-5690	George	Yellow

Union

Student-ID	F-Name	L-Name
145-67-6754	John	Brown
232-56-5690	George	Yellow
345-89-6580	Anne	Green
459-98-6789	Ted	Purple
342-88-9999	Rich	White

Relational (1970s)

- Intersection

CIS15-Roster

Student-ID	F-Name	L-Name
145-67-6754	John	Brown
232-56-5690	George	Yellow
345-89-6580	Anne	Green
459-98-6789	Ted	Purple

CIS52-Roster

Student-ID	F-Name	L-Name
342-88-9999	Rich	White
145-67-6754	John	Brown
232-56-5690	George	Yellow

Intersection

Student-ID	F-Name	L-Name
145-67-6754	John	Brown
232-56-5690	George	Yellow

Relational (1970s)

- Example of relational database
 - Oracle (Cloud) – Oracle SQL Plus, SQL Developer
 - Microsoft SQL Server – Microsoft SQL Server Management Studio
 - MySQL – phpMyAdmin, MySQL-Front
 - IBM DB2 (on Cloud)
 - Google Cloud SQL
 - SQL Azure
- Popular query (declarative) language
 - SQL (Structure Query Language)
 - XQuery (any data sources that can be view as XML)

MySQL - PhPMyAdmin

```
mysql> create table saveMovie (  
  -> id int not null auto_increment,  
  -> Name varchar(30) not null unique,  
  -> Object1 varchar(60),  
  -> Object2 varchar(60),  
  -> Object3 varchar(60),  
  -> Comment text,  
  -> primary key (id));
```

Query OK, 0 rows affected (0.00 sec)

```
mysql> describe saveMovie;
```

Field	Type	Null	Key	Default	Extra
id	int(11)		PRI	NULL	auto_increment
Name	varchar(30)		UNI		
Object1	varchar(60)	YES		NULL	
Object2	varchar(60)	YES		NULL	
Object3	varchar(60)	YES		NULL	
Comment	text	YES		NULL	

6 rows in set (0.00 sec)

```
mysql>
```

Source: <https://www.sqlshack.com/mysql-create-table-statement-with-examples/>

MySQL - PhPMyAdmin

```
mysql> select Host,User,Password from user;
+-----+-----+-----+
| Host      | User  | Password      |
+-----+-----+-----+
| localhost | root  | 6d0e487e6cbc827d |
| i5        | root  |                |
| 127.0.0.1 | root  |                |
| localhost |      |                |
| i5        |      |                |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> delete from user where User = 'root' and Password = '';
Query OK, 2 rows affected (0.00 sec)

mysql> select Host,User,Password from user;
+-----+-----+-----+
| Host      | User  | Password      |
+-----+-----+-----+
| localhost | root  | 6d0e487e6cbc827d |
| localhost |      |                |
| i5        |      |                |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Source: <http://www.linux-commands.eu/mysql-commands-usually-run-from-shell/>

MySQL - PhPMyAdmin

The screenshot displays the phpMyAdmin web interface. On the left is a sidebar with a tree view of databases and a search filter. The main area shows the 'Users overview' page with a table of users and their privileges.

Current Server: phpMyAdmin demo - MySQL

filter databases by name > >>

User	Host	Password	Global privileges	Grant	Action
☐ debian-sys-maint	localhost	Yes	ALL PRIVILEGES	No	Edit Privileges Export
☐ lucasbrasilconta	localhost	Yes	USAGE	No	Edit Privileges Export
☐ pma	localhost	Yes	USAGE	No	Edit Privileges Export
☐ root	127.0.0.1	No	ALL PRIVILEGES	Yes	Edit Privileges Export
☐ root	localhost	No	ALL PRIVILEGES	Yes	Edit Privileges Export
☐ root	pmademo	No	ALL PRIVILEGES	Yes	Edit Privileges Export
☐ test_user	localhost	--	USAGE	No	Edit Privileges Export
☐ toor	%	Yes	ALL PRIVILEGES	Yes	Edit Privileges Export

☐ Check All With selected: [Export](#)

[Add user](#)

[Remove selected users](#)

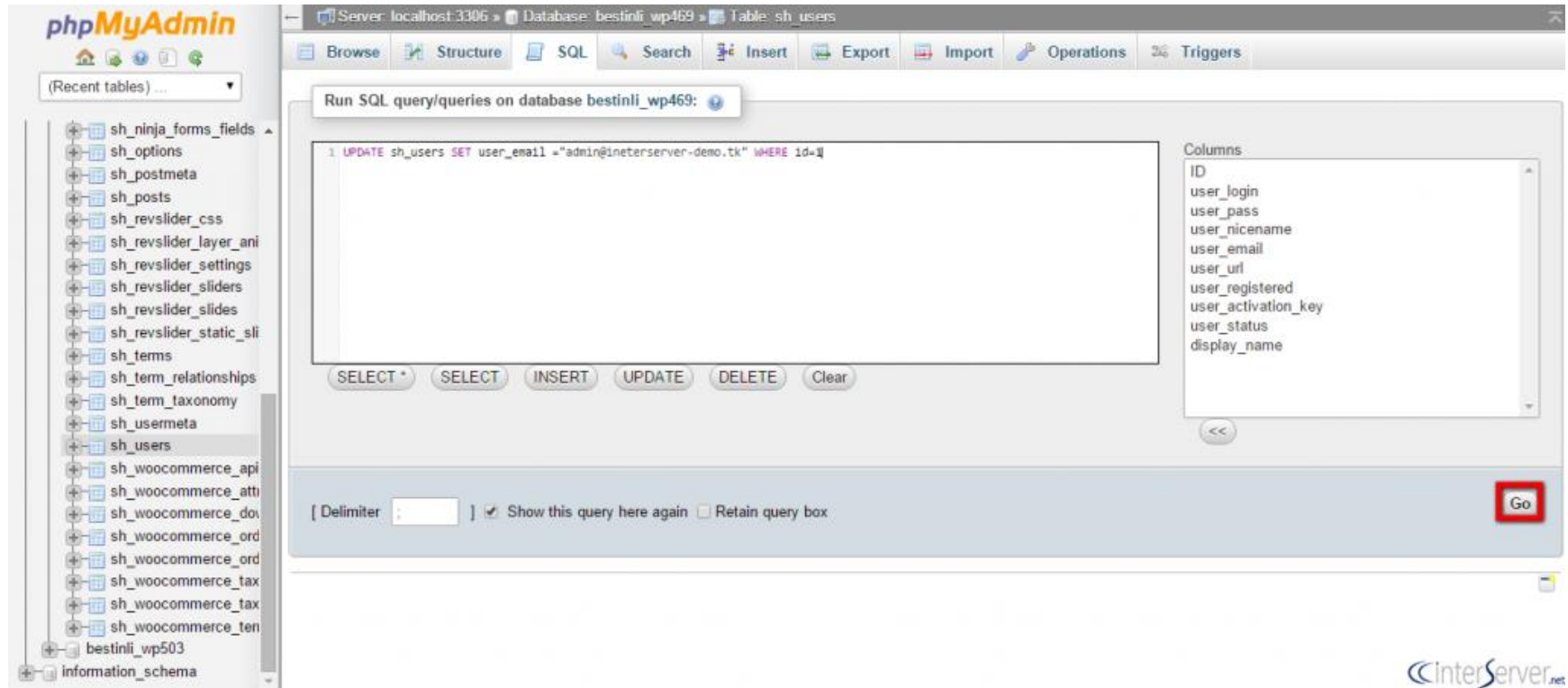
(Revoke all active privileges from the users and delete them afterwards.)
☐ Drop the databases that have the same names as the users.

[Go](#)

Note: phpMyAdmin gets the users' privileges directly from MySQL's privilege tables. The content of these tables may differ from the privileges the server uses, if they have been changed manually. In this case, you should [reload the privileges](#) before you continue.

Source: <https://www.phpmyadmin.net/try/>

MySQL - PhPMyAdmin



Source: <https://www.interserver.net/tips/kb/how-to-run-mysql-query-in-phpmyadmin/>

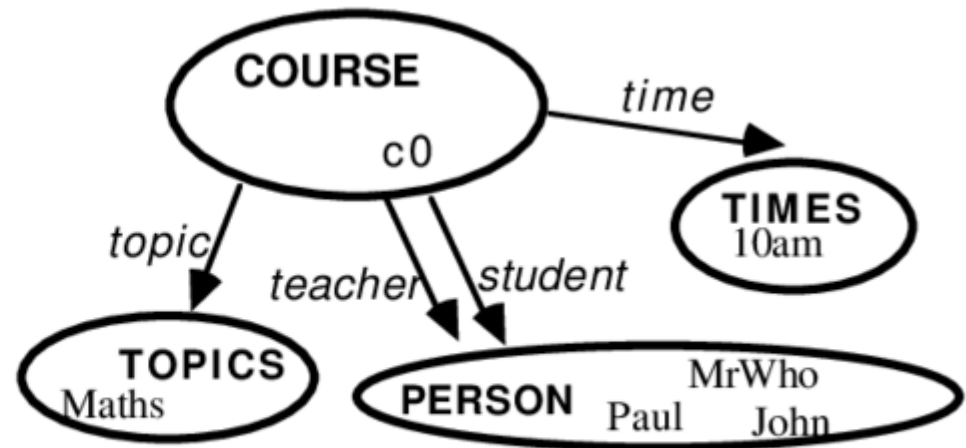
Object-oriented (1990s)

- Information is represented in the form of objects (not table-oriented)
- Object-relational database
 - Similar to relational database (table-like)
 - With additional object-oriented database model - objects, classes and inheritance are directly supported
- Most of current technologies in relational database are extended to support object-oriented database
 - IBM DB2
 - Oracle
 - Microsoft SQL Server

Object-oriented (1990s)

COURSE			
<i>topic</i>	<i>teacher</i>	<i>student</i>	<i>time</i>
Maths	MrWho	John	10am
Maths	Mr Who	Paul	10am
...			

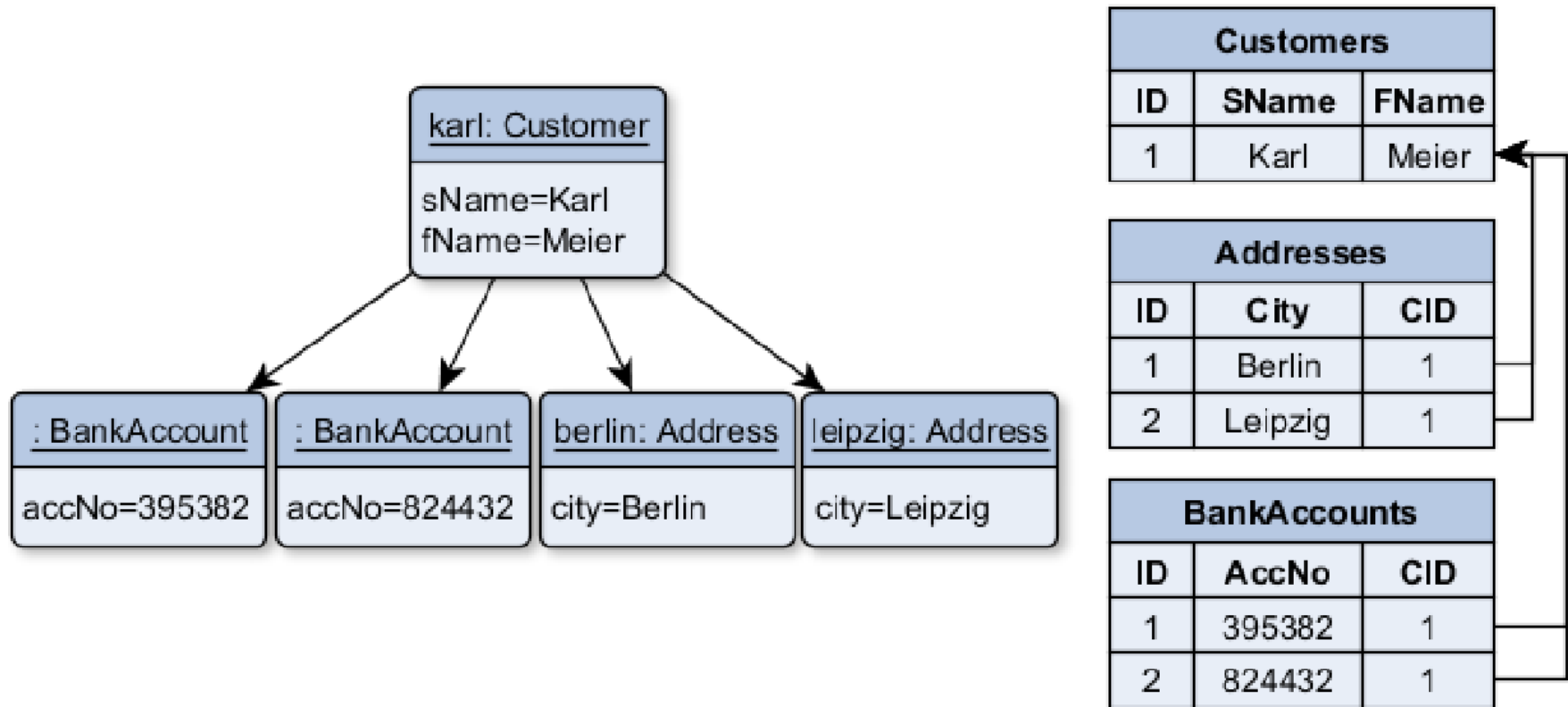
relational database approach



object-oriented database approach

Source: https://www.researchgate.net/figure/Object-Oriented-vs-Relational-Database_fig4_220643124

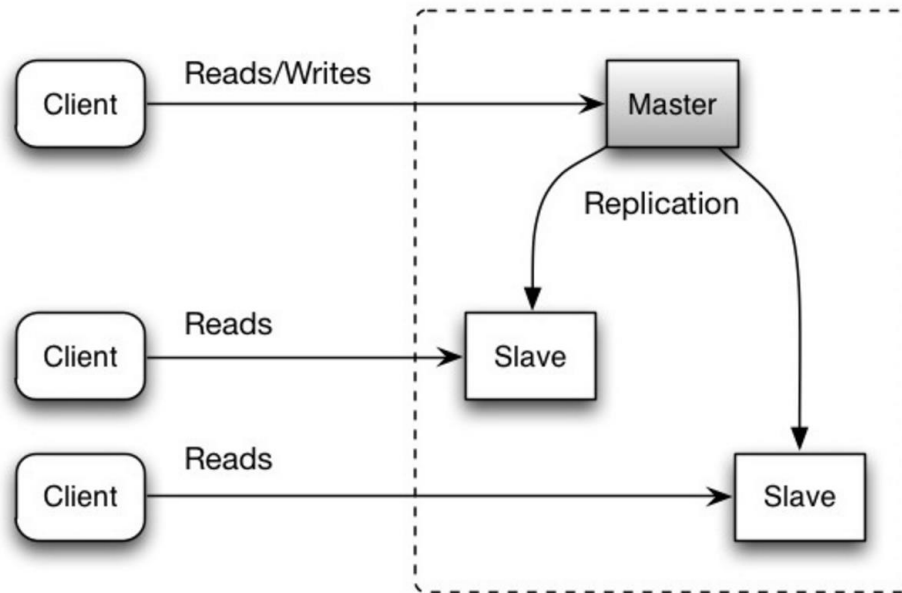
Object-oriented (1990s)



Source: <https://www.semanticscholar.org/paper/Evolution-of-Object-Oriented-Database-Systems-By-Alzahrani/6d1c233393fa9064957a75043cee480ce6a36478/figure/1>

SQL Scalability

- Master-slave
 - Critical reads may be incorrect as writes may not have been propagated down
 - Large data sets can pose problems when master needs to duplicate data to slaves



SQL Scalability

- Sharding (horizontal partitioning)
 - Partition aware
 - No longer have relationships or join across partitions

Original Table

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN
3	SELDA	BAGCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

Vertical Partitions

VP1

CUSTOMER ID	FIRST NAME	LAST NAME
1	TAEKO	OHNUKI
2	O.V.	WRIGHT
3	SELDA	BAGCAN
4	JIM	PEPPER

VP2

CUSTOMER ID	FAVORITE COLOR
1	BLUE
2	GREEN
3	PURPLE
4	AUBERGINE

Horizontal Partitions

HP1

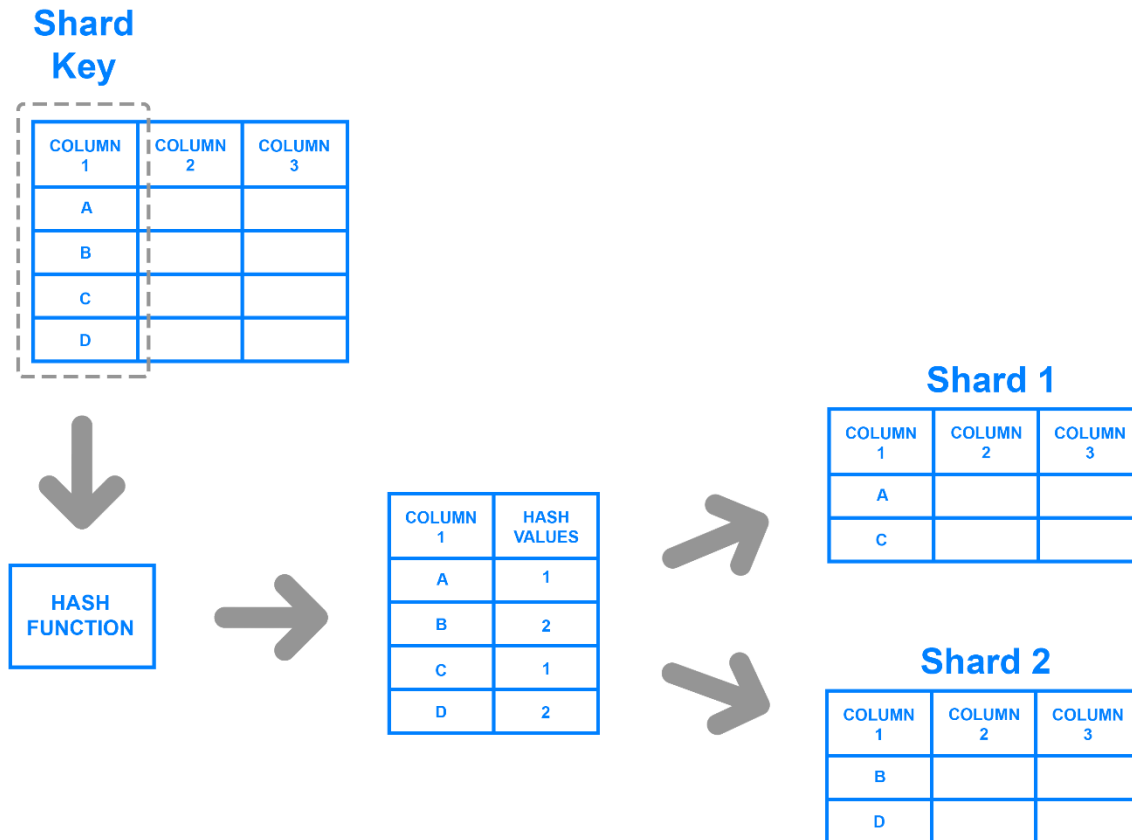
CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN

HP2

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
3	SELDA	BAGCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

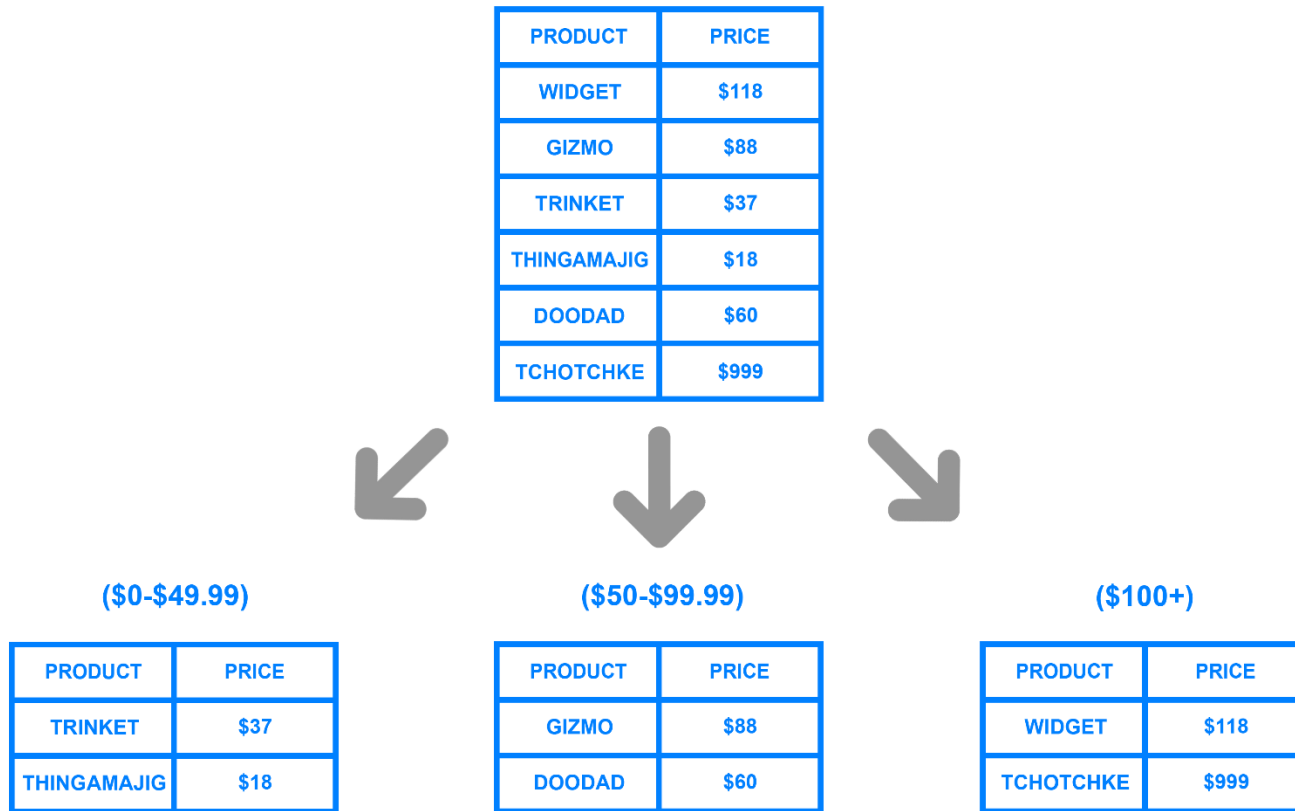
SQL Scalability

- Key Based Sharding



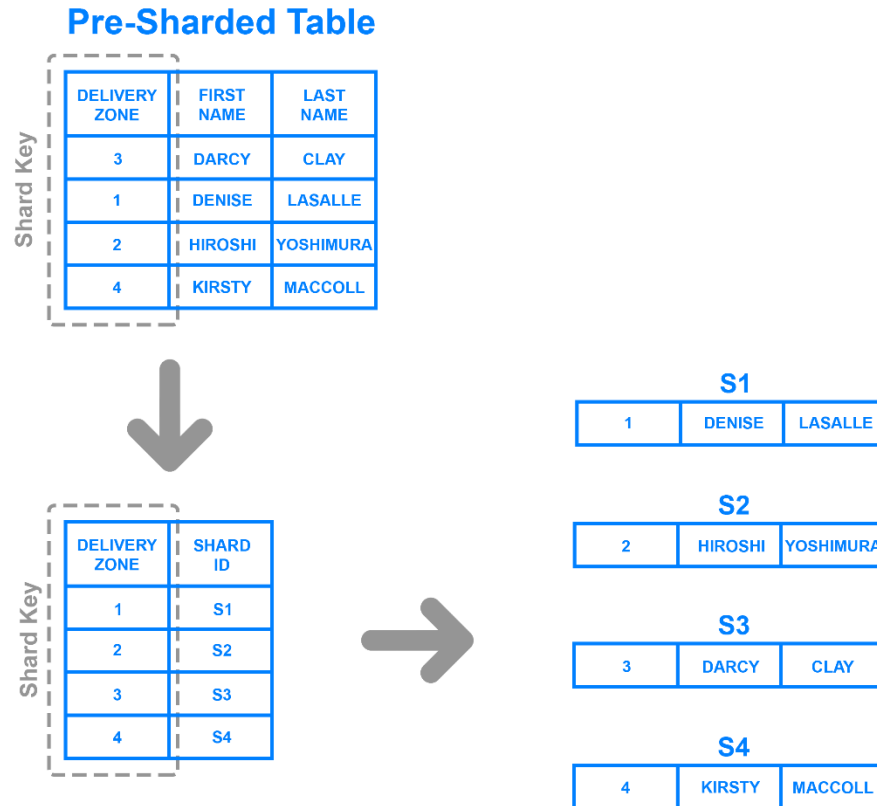
SQL Scalability

- Range Based Sharding



SQL Scalability

- Directory Based Sharding



NoSQL

- NoSQL – “Non SQL” or “Not only SQL”
- Class of non-relational data storage systems
- Do not require a fixed table schema nor do they use the concept of joins

Need of NoSQL

- RDBMS issue with scalability and not designed to be distributed
- Explosion of social media sites (Facebook, Twitter, Google etc.) with large data needs.
- Just as moving to dynamically-typed languages (Ruby/Groovy), a shift to dynamically-typed data with frequent schema changes

NoSQL Type

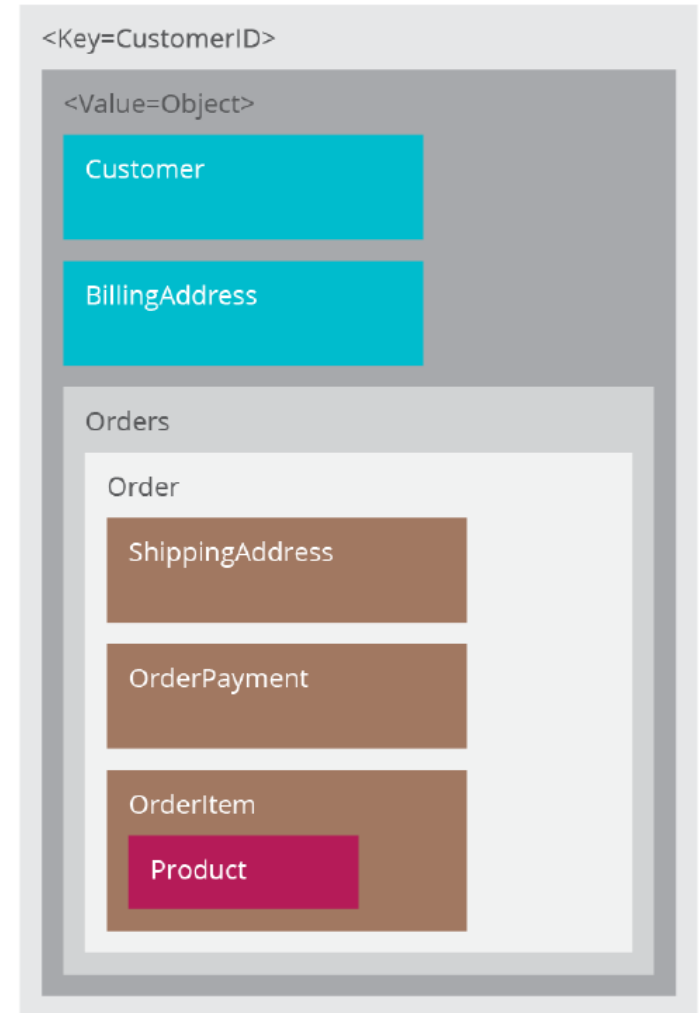
- Key Value pair based
- Column based
- Document based
- Graph based

Key Value Pair Based

- Design for processing dictionary which contains a collection of records having fields containing data
- Records are stored and retrieved using a key that uniquely identifies the record
- Common uses
 - Session information
 - User profiles
 - Preferences
 - Shopping cart
- Avoid when we need to query data having relationship between entities

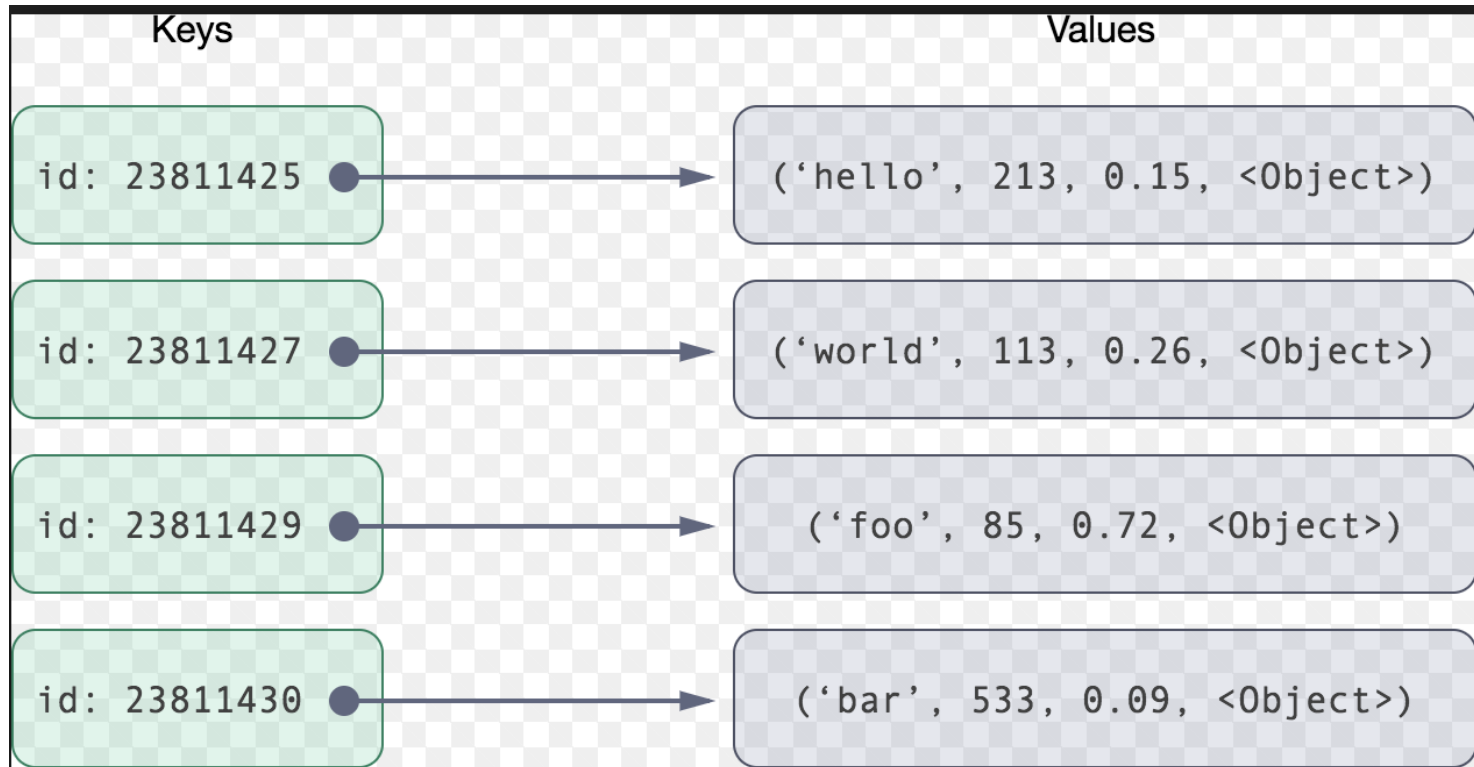
Key Value Pair Based

- Examples
 - Oracle NoSQL Database
 - Riak



Key Value Pair Based

- Oracle NoSQL Database (still can use simple SELECT Queries)



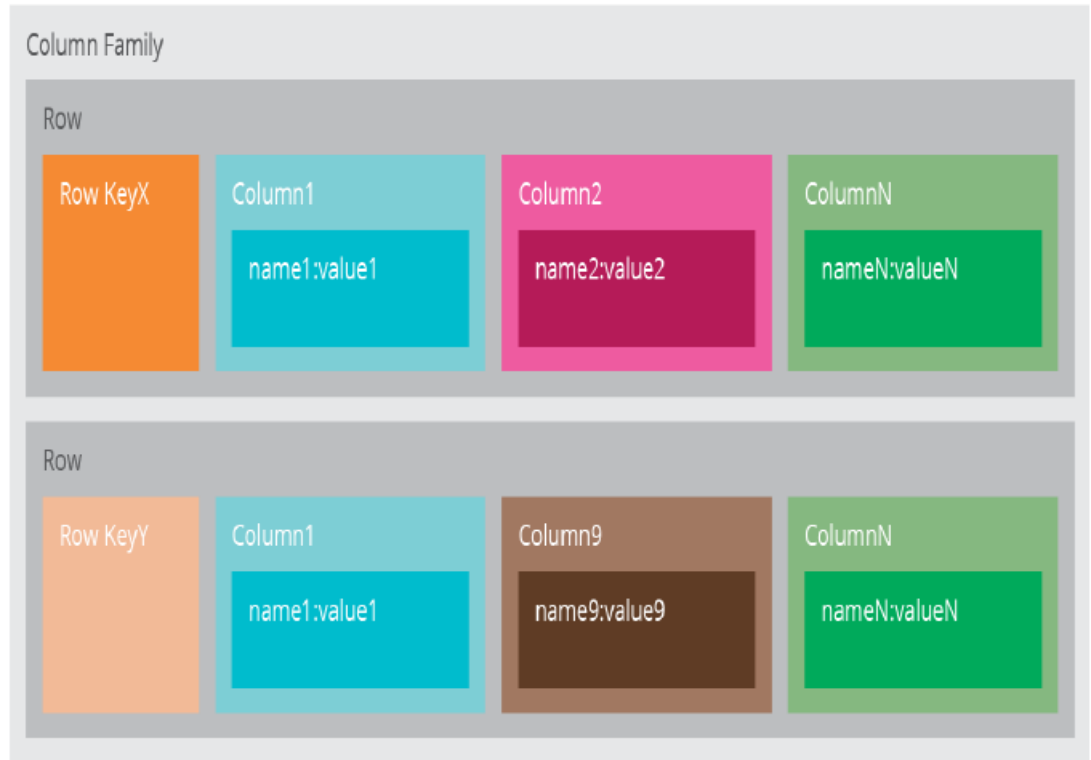
Source: <https://medium.com/@mark.rethana/introduction-to-nosql-databases-c5b43f3ca1cc>

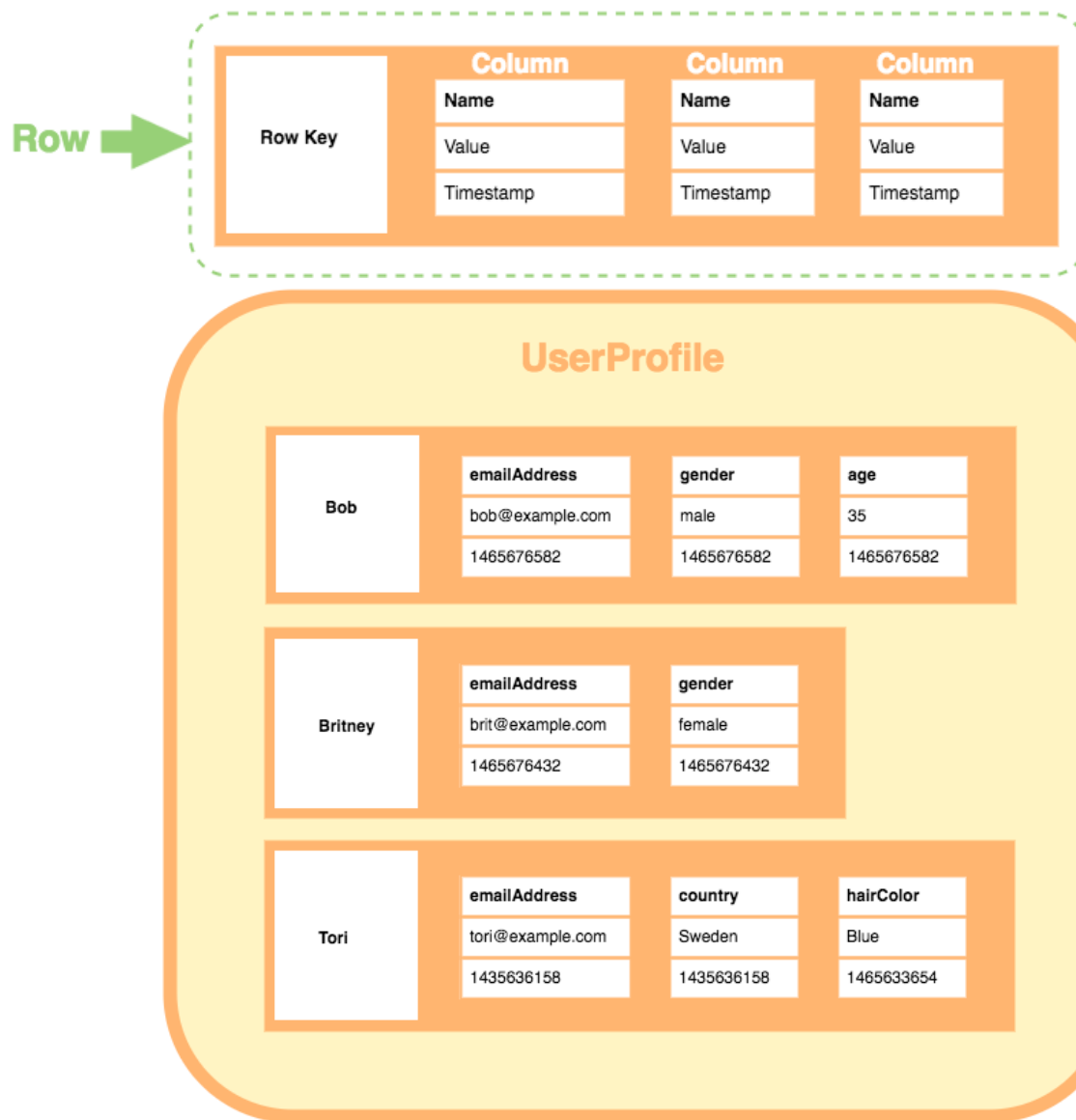
Column Based

- It stores data as Column families containing rows that have many columns associated with a row key
- Common use
 - Content management system
 - Blogging platforms
 - Log aggregation
- Avoid for systems that are in early development, changing query patterns

Column Based

- Examples
 - Cassandra
 - Hbase
 - Hypertable
 - Amazon DynamoDB





Source: <https://www.kdnuggets.com/2021/02/understanding-nosql-database-types-column-oriented-databases.html>

Column Based

- Cassandra (Column family)

partition key columns ...

101	email	name	tel	
	ab@c.to	otto	12345	
103	email	name	tel	tel2
	karl@a.b	karl	6789	12233
104	name			
	linda			

Column Based

- Cassandra (Column family: Tweets)

PRIMARY KEY
= msg_id

9990	author	body
	otto	Hello World!
9991	author	body
	linda	Hi, Otto

Column Based

- Cassandra (Column family: Timeline)

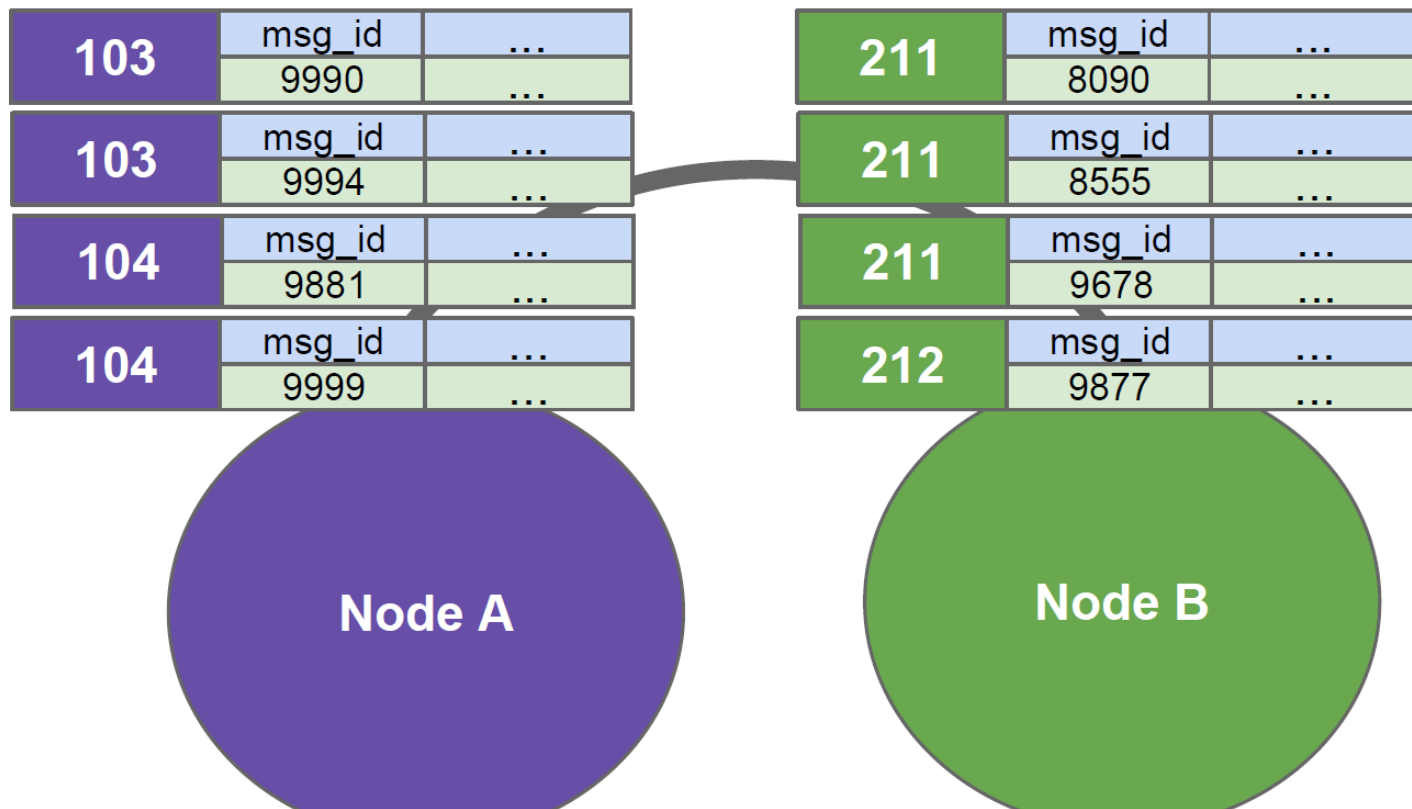
PRIMARY KEY = user_id + msg_id

partition key column

103	msg_id	author	body
	9990	otto	Hello World!
103	msg_id	author	body
	9991	linda	Hi @otto

Column Based

- Timeline is partitioned by user and locally clustered by msg



Column Based

- Comparison

RDBMS Data Design

Users Table

user_id	name	email
101	otto	o@t.to

Tweets Table

tweet_id	author_id	body
9990	101	Hello!

Followers Table

id	follows_id	followed_id
4321	104	101

Cassandra Data Design

Users Table

user_id	name	email
101	otto	o@t.to

Tweets Table

tweet_id	author_id	name	body
9990	101	otto	Hello!

Follows Table

user_id	follows_list
104	[101,117]

Followed Table

id	followed_list
101	[104,109]

Column Based

- Cassandra Query Language (CQL) – close to SQL

```
SELECT name, occupation FROM users WHERE userid IN (199, 200, 207);  
SELECT JSON name, occupation FROM users WHERE userid = 199;  
SELECT name AS user_name, occupation AS user_occupation FROM users;  
  
SELECT time, value  
FROM events  
WHERE event_type = 'myEvent'  
    AND time > '2011-02-03'  
    AND time <= '2012-01-01'  
  
SELECT COUNT (*) AS user_count FROM users;
```

Document Based

- Store and retrieve documents
- It stores documents in the value part of the key-value store
- Common uses
 - Content management systems
 - Blogging platforms
 - Web analytics
 - Real-time analytics
 - E-commerce application
- Avoid for the systems that need complex transactions scanning multiple operations or queries against varying aggregate structures

Document Based

- Examples
 - MongoDB
 - CouchDB
 - Orient DB
 - Raven DB

```
<Key=CustomerID>

{
  "customerid": "fc986e48ca6" ← Key
  "customer":
  {
    "firstname": "Pramod",
    "lastname": "Sadelage",
    "company": "ThoughtWorks",
    "likes": [ "Biking", "Photography" ]
  }
  "billingaddress":
  { "state": "AK",
    "city": "DILLINGHAM",
    "type": "R"
  }
}
```

Document Based

- MongoDB

```
anki — mongo — 80x28
> db.employee.updateMany({}, {$set: {salary: 50000}})
{ "acknowledged" : true, "matchedCount" : 3, "modifiedCount" : 3 }
> db.employee.find().pretty()
{
  "_id" : ObjectId("5e49177592e6dfa3fc48dd73"),
  "name" : "Sonu",
  "age" : 26,
  "branch" : "CSE",
  "department" : "HR",
  "salary" : 50000
}
{
  "_id" : ObjectId("5e49813692e6dfa3fc48dd74"),
  "name" : "Rohit",
  "age" : 20,
  "branch" : "CSE",
  "department" : "Development",
  "salary" : 50000
}
{
  "_id" : ObjectId("5e49813692e6dfa3fc48dd75"),
  "name" : "Mohit",
  "age" : 26,
  "branch" : "ECE",
  "department" : "Development",
  "salary" : 50000
}
>
```

Source: <https://www.geeksforgeeks.org/mongodb-update-multiple-documents-using-mongoshell/>

Document Based

- MongoDB – Query Documents in a collection

```
{ status: { $in: [ "A", "D" ] } }
```

```
_id: ObjectId("5a1468c1a769896e78222e5f")  
item: "journal"  
qty: 25  
> size: Object  
status: "A"
```

```
_id: ObjectId("5a1468c1a769896e78222e60")  
item: "notebook"  
qty: 50  
> size: Object  
status: "A"
```

```
_id: ObjectId("5a1468c1a769896e78222e61")  
item: "paper"  
qty: 100  
> size: Object  
status: "D"
```

```
_id: ObjectId("5a1468c1a769896e78222e62")  
item: "planner"  
qty: 75  
> size: Object  
status: "D"
```

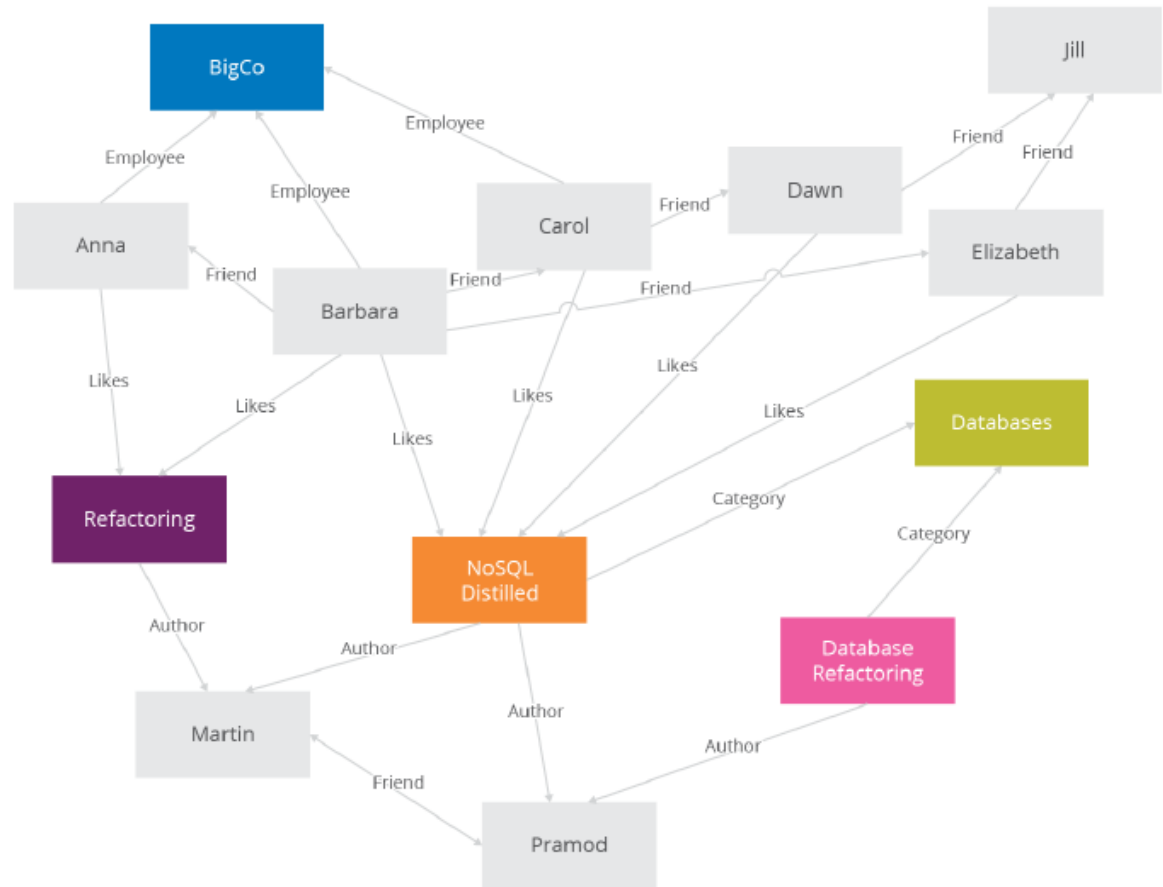
```
_id: ObjectId("5a1468c1a769896e78222e63")  
item: "postcard"  
qty: 45  
> size: Object  
status: "A"
```


Graph Based

- Store entities and relationships between entities as nodes and edges of a graph
- Traversing the relationships is very fast as it is not calculated at query time but actually persisted as relationship
- Common uses
 - Social networks
 - Spatial data
 - Routing information

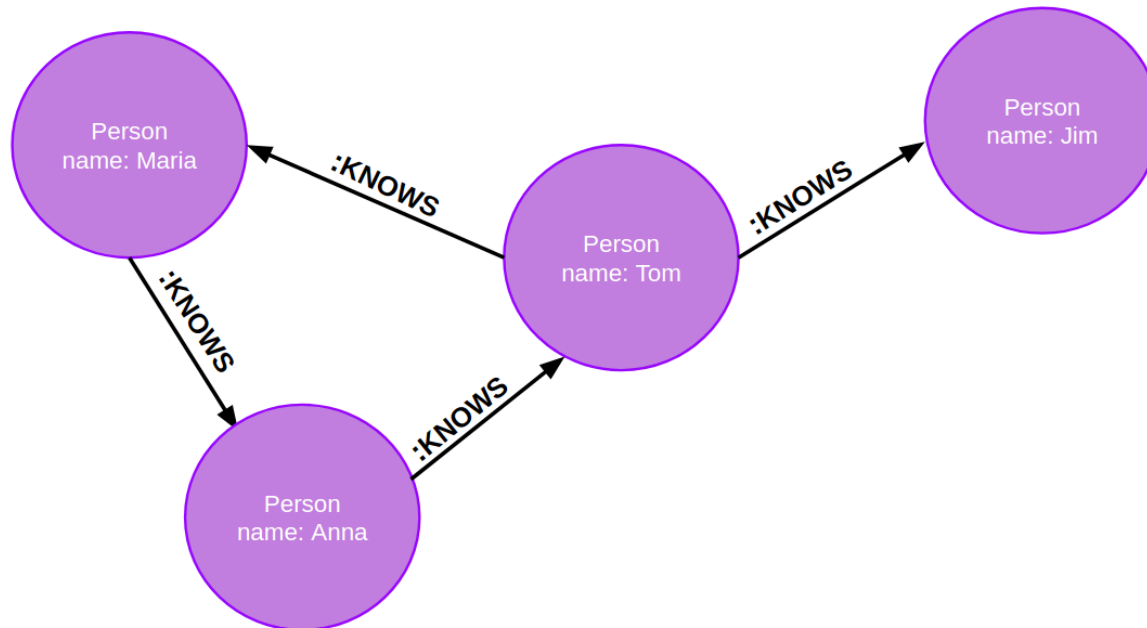
Graph Based

- Examples
 - Neo4J
 - Infinite Graph
 - OrientedDB
 - FlockDB



Graph Based

- Neo4J



Source: <https://terminusdb.com/blog/2020/09/24/graph-databases-terminusdb-vs-neo4j/>

Graph Based

- Neo4J – Cypher Query Language

```
MATCH (:Person {name: 'Tom Hanks'})-[ :DIRECTED]->(movie:Movie)
RETURN movie
```

Run Example

RESULTS



The graph shows a single teal-colored circular node with the text 'That Thing You Do' inside it.

```
//cleaner printed results with aliasing
MATCH (tom:Person {name:'Tom Hanks'})-[rel:DIRECTED]-(movie:Movie)
RETURN tom.name AS name, tom.born AS 'Year Born', movie.title AS title, movie.released AS 'Year Released'
```

Run Example

RESULTS Close Results

name	Year Born	title	Year Released
Tom Hanks	1956	<i>That Thing You Do</i>	1996

Exercise 11.1

- Discuss pro and con of each database structure as well as discuss situation in which each structure is appropriate.

Exercise 11.2

- Create MySQL database of products having following fields:
 - Product id
 - Product name
 - Price per unit
 - Product type (food, drink, etc.)
- Create a webpage having a form to list all product by filtering product type.